

TRABAJO PRÁCTICO

RIR-API.

Una API REST para el cálculo de parámetros acústicos según la norma ISO 3382.

Primero, el enunciado.

Presentación maestra de la cátedra con el marco teórico del TP: acústica de salas, parámetros ISO 3382, flujo del proyecto.

docs.google.com/presentation/d/1XJAI0wFRRS6IaVops3jCAcfdRxvMJyQs_mIetzehh1c

Vamos a recorrerla juntos. Cuando terminemos, volvemos acá para aterrizar los detalles de la cursada.

¿Qué van a construir?

RIR-API: una API REST en Python (FastAPI) que calcula parámetros acústicos a partir de respuestas al impulso, siguiendo la norma UNE-EN ISO 3382.

EL SISTEMA PROCESA

- Generación de señales de excitación (ruido rosa, sine sweep)
- Grabación y procesamiento de respuestas al impulso
- Cálculo de parámetros: **EDT, T20, T30, T60, D50, C80**

CONSUMIBLE DESDE

Frontend web · script Python · otra aplicación · curl. Cualquier cliente HTTP.

Precisión requerida: resultados dentro de ± 0.5 s respecto a software comercial.

Saben hacer esto.

DESARROLLO

- Diseñar e implementar una API REST modular
- Arquitectura en capas con validación Pydantic
- Testing, linting, CI/CD
- Packaging y deploy a producción

PROCESAMIENTO DE SEÑALES

- Implementar generación, filtrado, deconvolución
- Interpretar y aplicar ISO 3382 · IEC 61260
- Validar resultados contra referencia comercial
- Documentar y presentar profesionalmente

Cuatro entregas incrementales.

MILESTONE	QUÉ ENTREGAN	ENTREGA	TAG	NOTA
M0 El Plano	Arquitectura · repo · issues	7 Oct	—	seguimiento
M1 Generación	Ruido rosa · sine sweep · record	28 Oct	v0.1.0	seguimiento
M2 Procesamiento	Filtros · síntesis RI · deconvolución	4 Nov	v0.2.0	seguimiento
M3 Producto	API REST · validación · demo + oral	18 Nov	v1.0.0	con nota

Cada milestone se entrega con un **tag de GitHub** en la fecha indicada. M0, M1 y M2 son de **seguimiento**: muestran su avance y reciben feedback por Slack. **Solo M3 + la oral llevan nota.**

El Plano.

Antes de escribir código, planifican. Arquitectura, grupo, issues, branching.

ENTREGAN

- README con integrantes y roles
- Diagrama de arquitectura (Mermaid o draw.io)
- ≥ 10 GitHub Issues con labels y asignación
- Proyecto ejecutable · endpoint `/health`
- Branching strategy documentada

PRESENTACIÓN DEDICADA

El detalle completo de M0 — incluyendo checklist y ejemplos — está en una presentación aparte:

→ [m0_presentacion.html](#)

Generación de señales.

Generar las señales de excitación que usarán para medir respuestas al impulso.

FUNCIONES REQUERIDAS

<code>generar_ruido_rosa(duracion, fs)</code>
<code>generar_sine_sweep(f1, f2, duracion, fs)</code>
<code>reproducir_y_grabar(signal, fs, duracion)</code>

DETALLES TÉCNICOS

- **Ruido rosa:** algoritmo Voss-McCartney · espectro -3 dB/octava
- **Sine sweep:** logarítmico + filtro inverso
- **Reproducción/grabación:** simultánea con `sounddevice`

Procesamiento de la RI.

Obtienen la respuesta al impulso y la preparan para el análisis acústico.

FUNCIONES REQUERIDAS

- `cargar_audio(ruta)`
- `sintetizar_ri(t60_por_banda, fs, duracion)`
- `obtener_ri_desde_sweep(grabacion, filtro_inverso)`
- `filtro_octava(signal, fc, fs, orden)`
- `a_escala_log(signal)`

TEMAS CLAVE

- Deconvolución por FFT
- Filtros de banda de octava **IEC 61260** (Butterworth)
- Síntesis de RI con T60 conocido — para validación

El producto.

API REST completa expuesta como endpoints · validación en el README · demo en vivo.

FUNCIONES DE ANÁLISIS

- suavizar_signal(signal, ventana)
- integral_schroeder(ri)
- regresion_lineal(x, y)
- calcular_parametros_acusticos(ri, fs)
 - EDT, T10, T20, T30, T60, D50, C80 por banda
- metodo_lundeby(ri, fs) extra

LA API

Endpoints FastAPI que exponen toda la funcionalidad de M1, M2 y M3. Documentación automática con Swagger UI.

PRESENTACIÓN ORAL · 25 MIN

Demo en vivo · decisiones de diseño · validación contra software comercial · reflexiones.

Stack tecnológico.

LENGUAJE · API

- Python 3.10+
- FastAPI
- Pydantic
- Uvicorn · ASGI

DSP · AUDIO

- NumPy · SciPy
- soundfile · sounddevice
- matplotlib

DEV · DEPLOY

- uv (paquetes)
- pytest · ruff
- Git · GitHub Actions

Todas son herramientas de uso profesional. Si aprenden este stack, lo llevan a cualquier trabajo de ingeniería de software.

Grupos de 3 a 4 integrantes.

Cada integrante debe tener un rol definido y contribuciones verificables en el historial de Git.

ROLES SUGERIDOS

- **Señales** — generación y grabación
- **Procesamiento** — filtros, deconvolución, RI
- **Testing · CI** — pytest, GitHub Actions
- **Documentación** — README, docs de la API, validación

NO DECORATIVO

Se evalúa el historial de Git para verificar contribuciones equitativas. **Diferencias significativas pueden resultar en notas individuales distintas dentro del mismo grupo.**

Distribución de notas.

COMPONENTE	FECHA	PESO
M3 Producto final	18 Nov	60%
Presentación oral	18 Nov	40%
M0 M1 M2 Seguimiento	7 Oct · 28 Oct · 4 Nov	sin nota
Log de desarrollo con IA	Todo el TP	obligatorio

Nota del TP = 60% M3 + 40% oral. Mínimo para aprobar: 4/10 en M3 y 4/10 en la nota del TP. El TP es el 20% de la nota de la materia.

Cinco criterios.

M3 se evalúa con estos cinco criterios. En M0, M1 y M2 los usamos para darles feedback (sin nota).

- **Funcionalidad** · 30% — las funciones hacen lo que deben
- **Calidad de código** · 20% — estructura, docstrings, type hints, `ruff`
- **Testing** · 20% — cobertura, casos de borde, CI

- **Documentación** · 15% — README, ejemplos, diagramas
- **Prácticas Git** · 15% — ramas, commits, PRs, tags

M0 tiene su propio checklist de entrega.

Validación y oral.

VALIDACIÓN EN EL README

- **No hay informe escrito** este cuatrimestre
- Sección **Validación** en el README del repo
- Tabla RIR-API vs. software comercial (EDT, T20, T30, C80 por banda)
- Gráficas: curva de decaimiento, filtros, validación · diagrama de arquitectura

PRESENTACIÓN ORAL · 18 NOV

- **Duración:** 20 min + 5 min de preguntas
- Demo en vivo de la API (Swagger UI, requests)
- Decisiones de diseño, validación, reflexión
- Todos los integrantes participan

Log de desarrollo con IA.

Archivo `AI_LOG.md` en la raíz del repo, actualizado con cada milestone.

CADA ENTRADA DOCUMENTA

- Fecha y milestone
- Herramienta usada (ChatGPT, Claude, Copilot, etc.)
- Prompt / consulta realizada (resumido)
- Resultado obtenido (resumido)
- Evaluación: ¿fue útil? ¿qué se modificó? ¿qué se aprendió?

LO QUE BUSCAMOS

- Honestidad** — documentar sin omisiones
- Reflexión** — evaluar críticamente las respuestas
- Aplicación** — adaptar las sugerencias al contexto

El uso de IA está **permitido y fomentado** como herramienta de aprendizaje. Lo que importa es la honestidad y la reflexión crítica. Sin `AI_LOG.md` , M3 no está completo.

Políticas.

ENTREGAS TARDÍAS

- M0, M1 y M2: si se atrasan, pierden el feedback a tiempo — y lo pagan en M3
- M3 fuera de fecha: pasa a **recuperatorio**
- Recuperatorio: una semana adicional · nota máxima 7/10

INTEGRIDAD ACADÉMICA

- El código debe ser **original del grupo**
- Copiar de otro grupo sin atribución = **desaprobación del TP** para ambos grupos
- Las funciones especificadas deben implementarse · no reemplazarse por llamadas a librerías de alto nivel que oculten la lógica

La cátedra ya hizo este recorrido.

Implementación de referencia en producción: pueden explorarla, usarla como guía, y compararla con su propia solución.

API — BACKEND

rir-api.onrender.com/docs

Swagger UI · documentación interactiva de endpoints y schemas.

FRONTEND — DEMO

Aplicación web que consume la API. **Se mostrará en clase como demo.**

Es un ejemplo concreto de cómo llevar una idea técnica a un producto funcional.

La API es **guía, no solución**. Cada grupo desarrolla su propia implementación.

Recursos.

DEL TP

- [Presentación conceptual \(Google Slides\)](#)
- [MO · presentación dedicada](#)
- Consigna: `trabajo_practico/README.md`
- Rúbrica: `trabajo_practico/rubrica.md`

API DE REFERENCIA

- [Swagger UI](#)
- [ReDoc](#)

NORMATIVA

- ISO 3382-1:2009
- IEC 61260-1:2014

HERRAMIENTAS

- [FastAPI](#)
- [Pydantic](#)
- [uv](#)
- [pytest](#)

PRÓXIMO PASO

Arrancamos con M0.

→ [m0_presentacion.html](#)